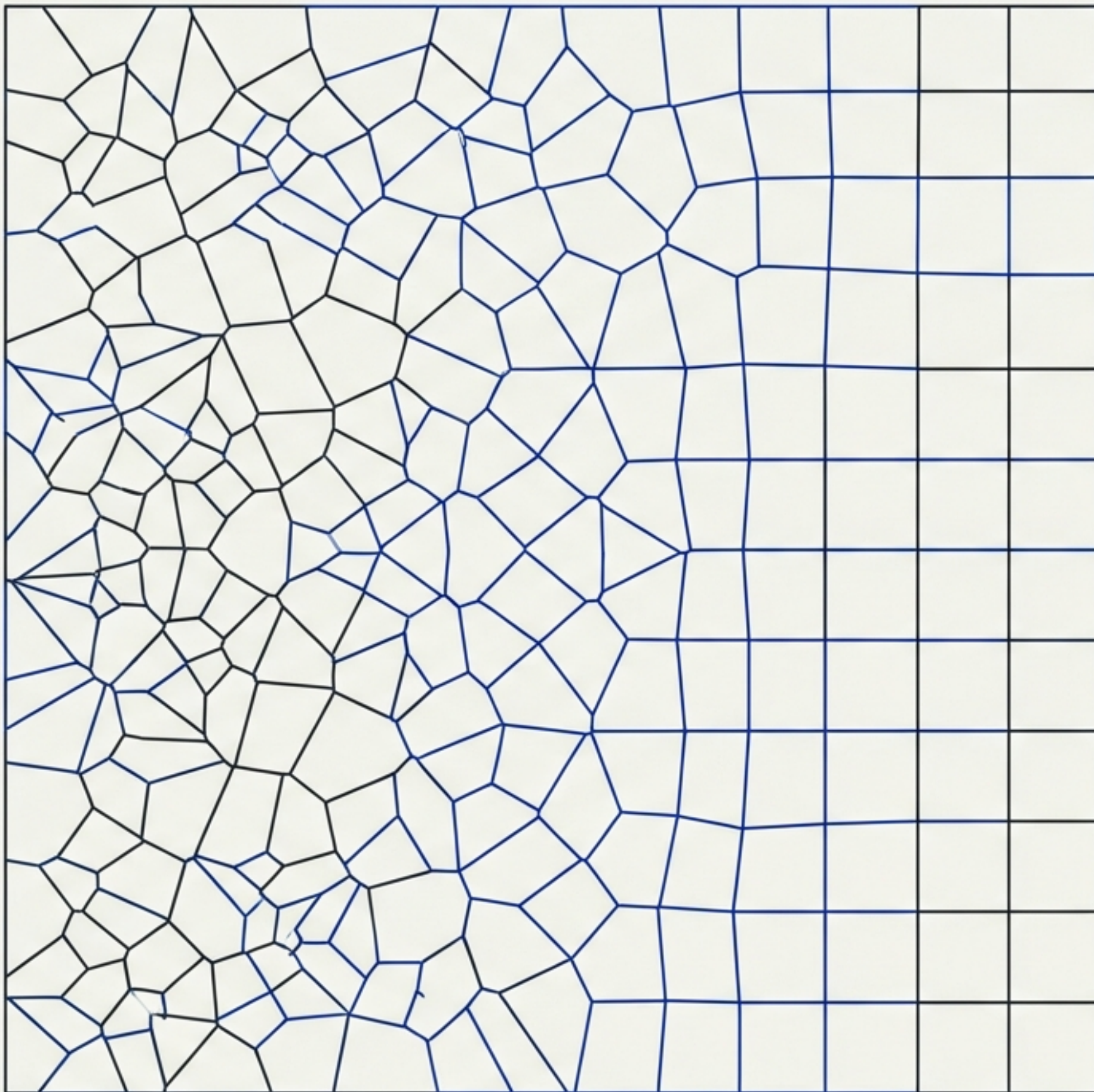




Architecture Strategy: Schema Visualization & Folder Integrity

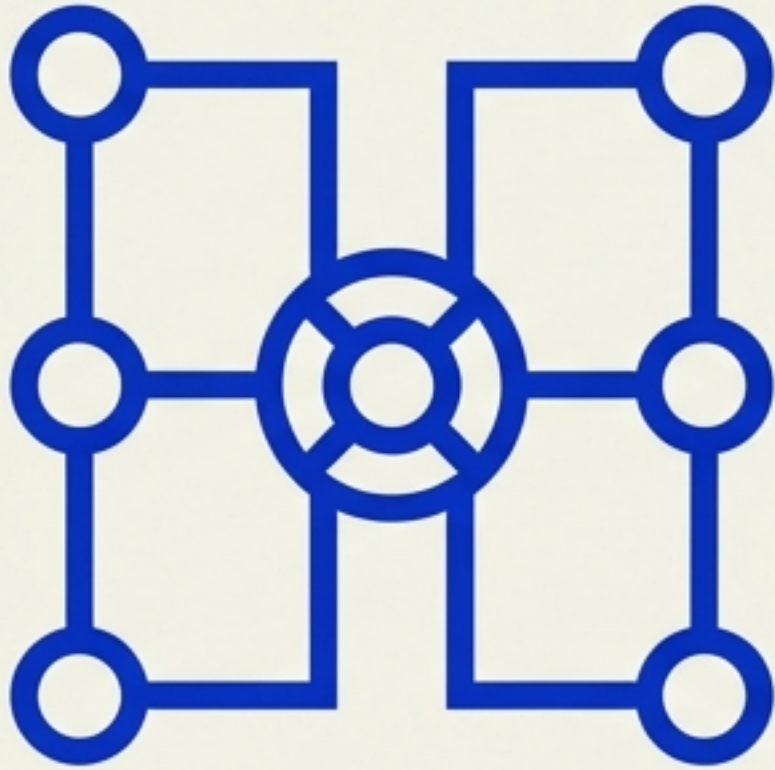
Proposal for Review | Draft v0.1 | 2026-02-01

Based on Brief-003

Related to MGraph-DB (Brief-001)

Related to Hierarchical Folders (Brief-002)

Governing complexity through three interconnected mechanisms



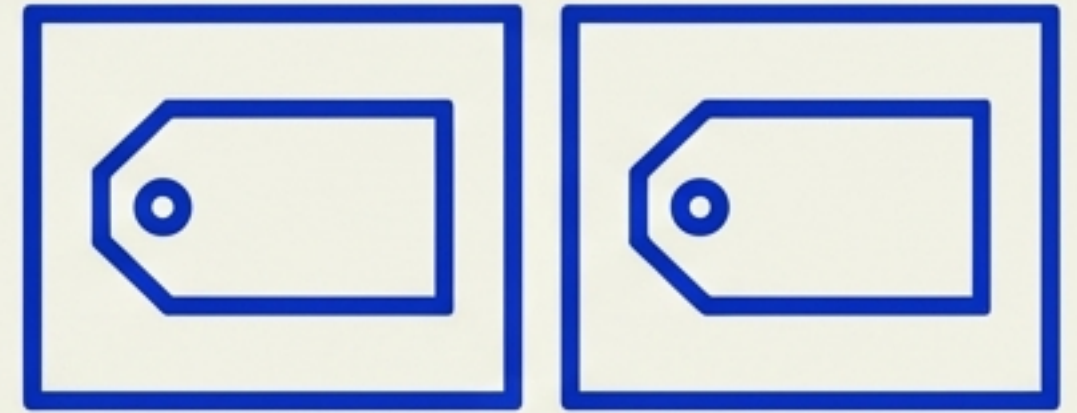
1. Schema Visualization

Curating and viewing the “meta-schema” to define which issue types can link to which, and with what verbs.



2. Folder-as-Schema

Ensuring the physical folder structure mirrors the logical graph structure, turning every folder into a ‘mini-database’.



3. Label Uniqueness Rules

Enforcing strict uniqueness within folders while allowing duplicates across parallel folders (local scope).

Value Proposition: These features enable better schema governance, creating ‘movable work units’ and ensuring scalable organization.

Without a meta-schema, relationships degrade into a spaghetti graph

In complex systems (like JIRA at scale), the lack of constraint leads to inevitable entropy:

- Proliferation of Types: Endless issue types (bug, task, feature, epic, story, subtask, incident...).
- Link Ambiguity: A web of link types (blocks, depends-on, relates-to, implements) with no clear rules.
- The Result: There is no clear understanding of which types can connect to which. The valid relationships are lost, and the schema becomes unmaintainable.



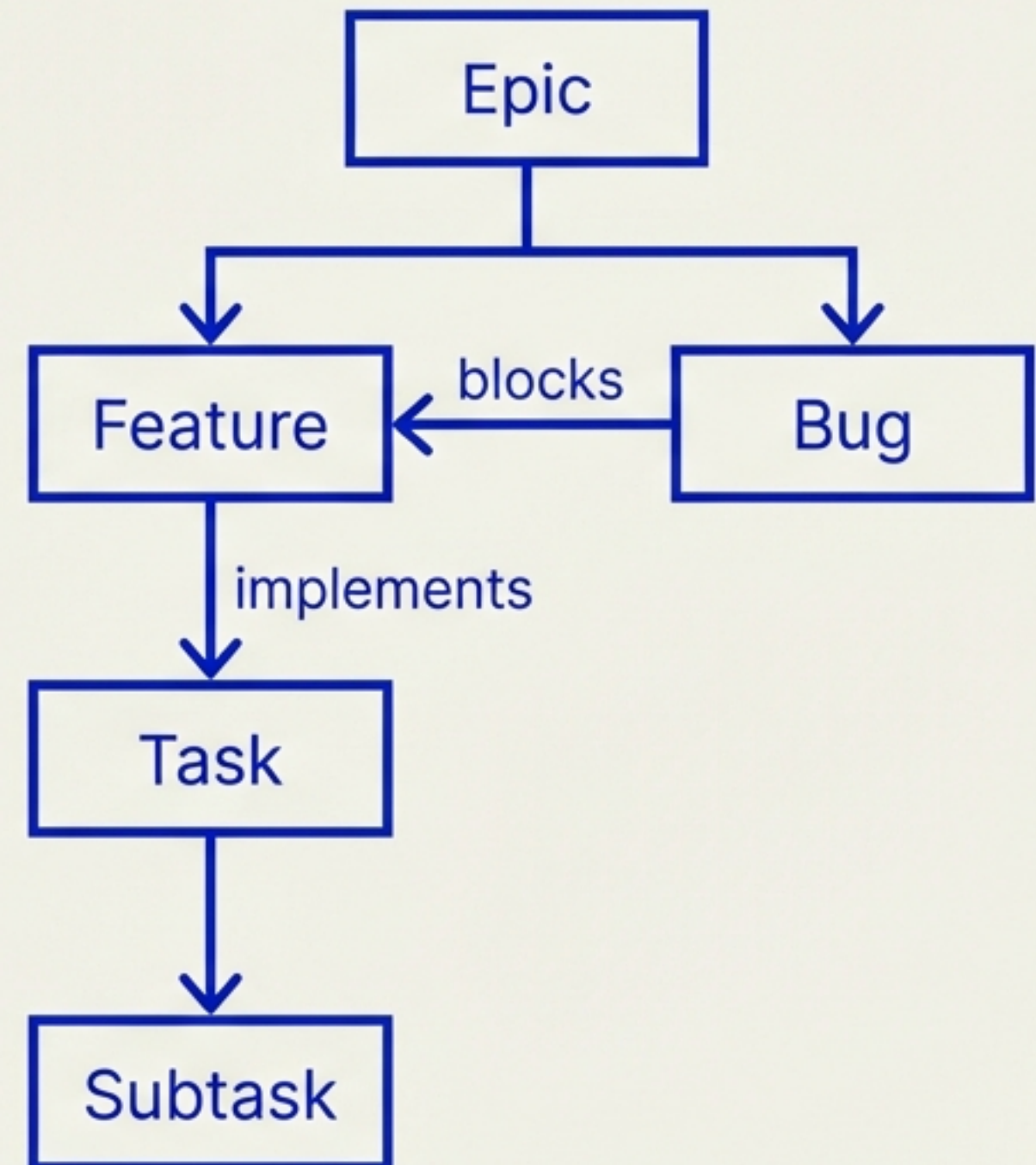
FIG 1.0: Current State: Unconstrained Graph

The Solution: Visualising the Meta-Schema

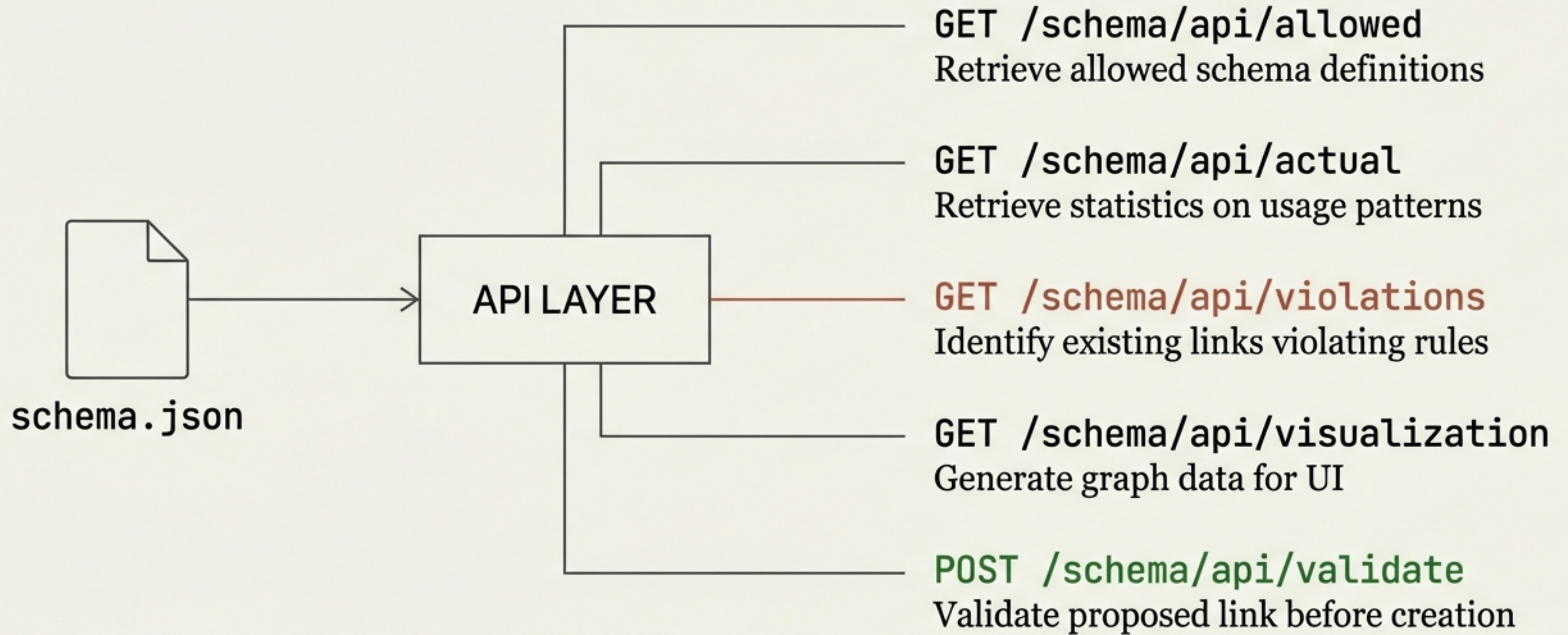
We solve entropy by explicitly defining the rules of the road in a configuration file.

- The Meta-Schema: A map that strictly defines valid relationships.
- Visualisation: The system renders this configuration as a clean, directed graph, allowing architects to curate system logic.

schema.json

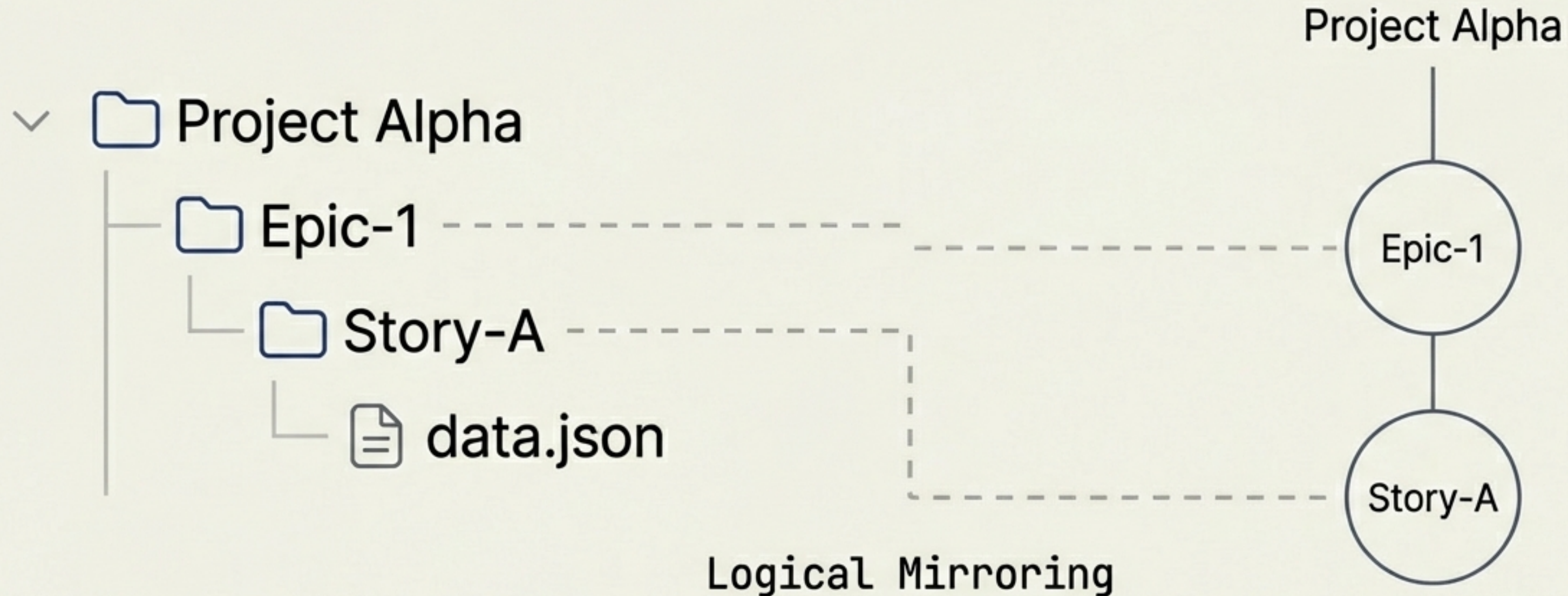


Five API endpoints to enforce and monitor schema integrity



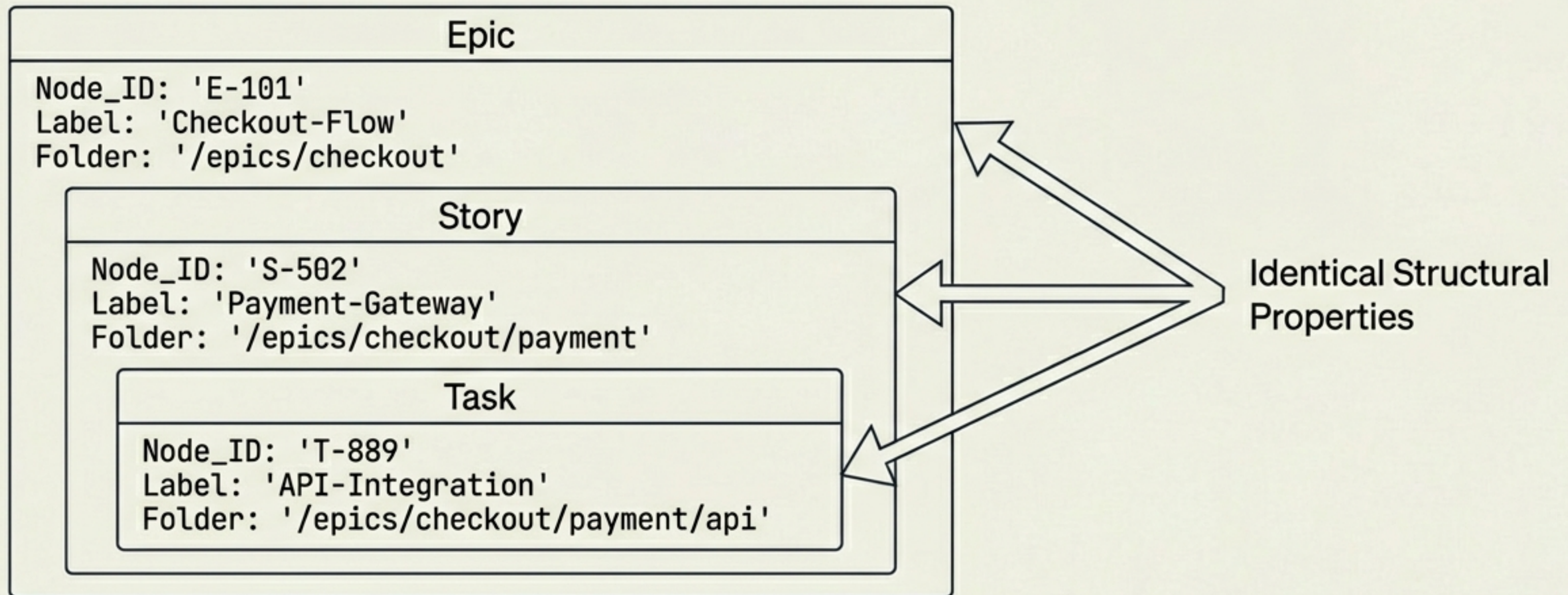
The Principle: Each folder is a self-contained mini-database

The physical file structure is not arbitrary; it mirrors the graph structure. Each folder fundamentally represents a type of node.



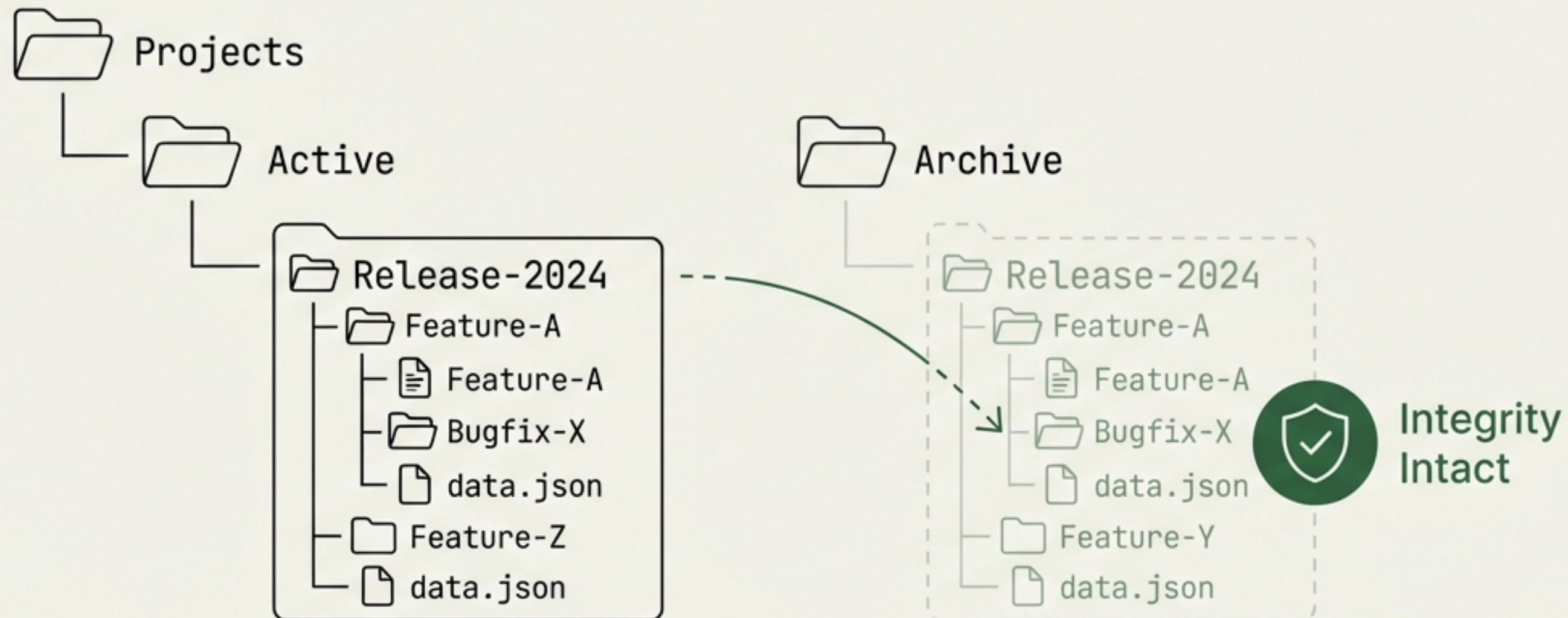
The schema is fractal and self-similar at every level

The system scales because the atomic unit—the Issue—follows a consistent pattern regardless of depth. An 'Epic' containing a 'Story' looks structurally identical to a 'Story' containing a 'Task'.



Self-contained folders become movable work units

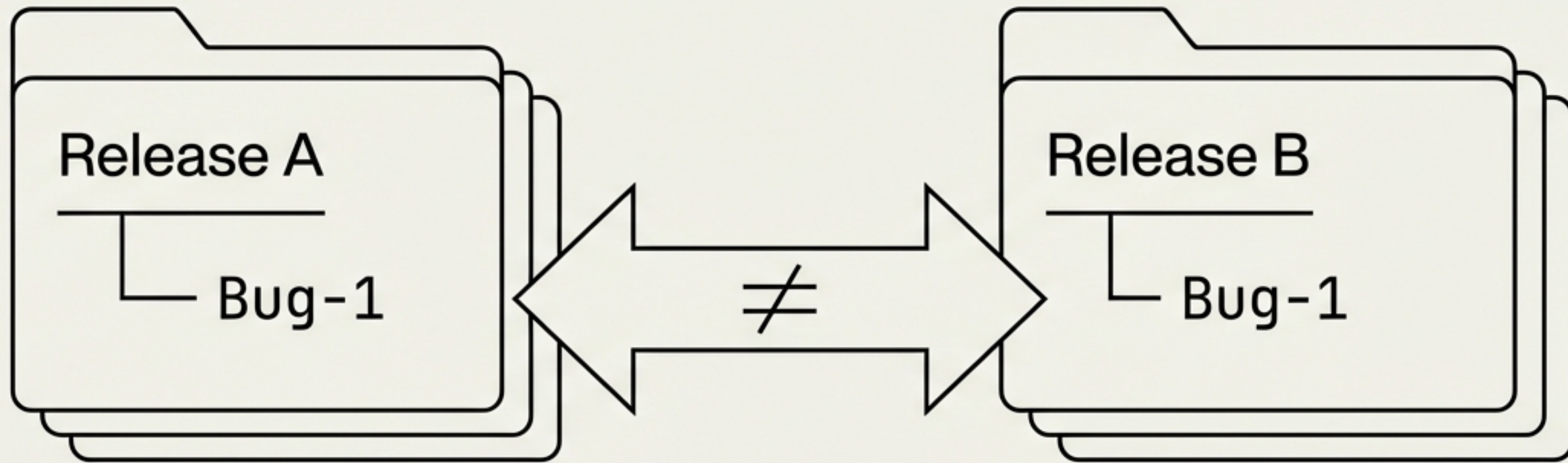
Because folders function as mini-databases, they can be relocated without breaking internal integrity. A “Release” folder containing features can be moved to “Archive” carrying its data and structure with it.



Swiss Technical Editorial

Labels are unique locally, but may duplicate globally

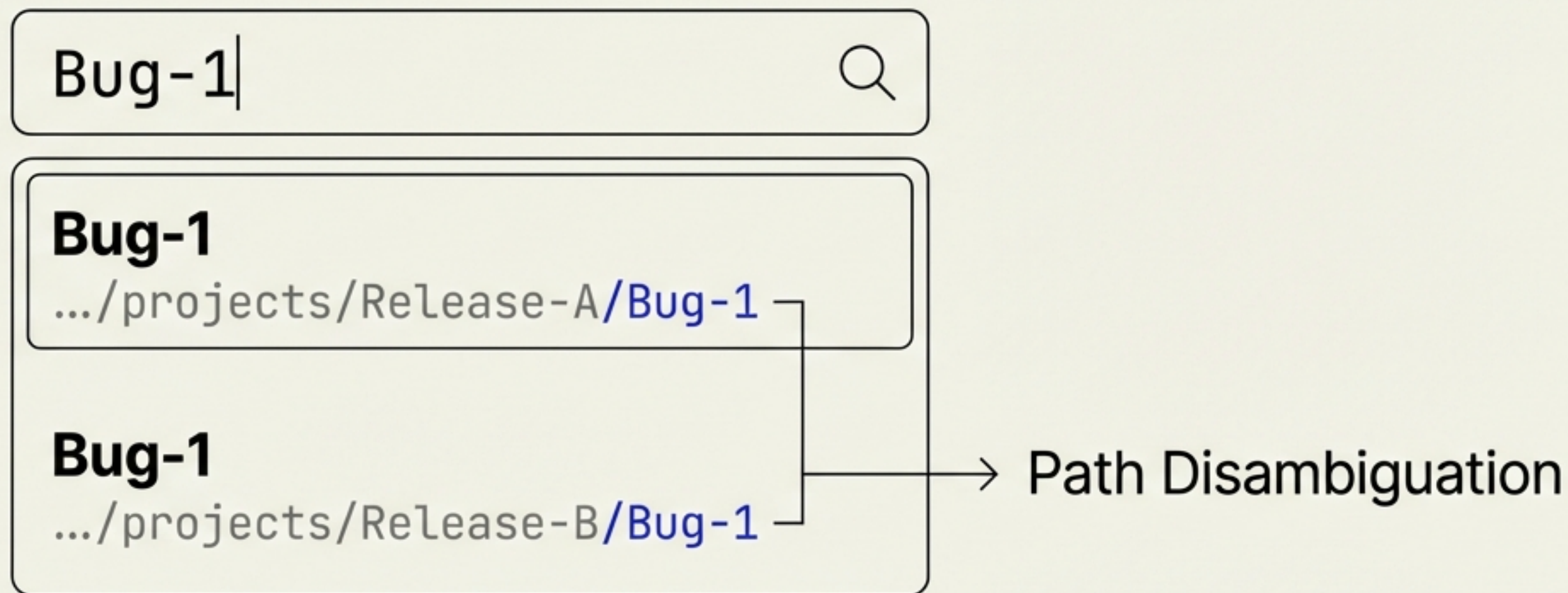
To support parallel work streams, we enforce Local Uniqueness



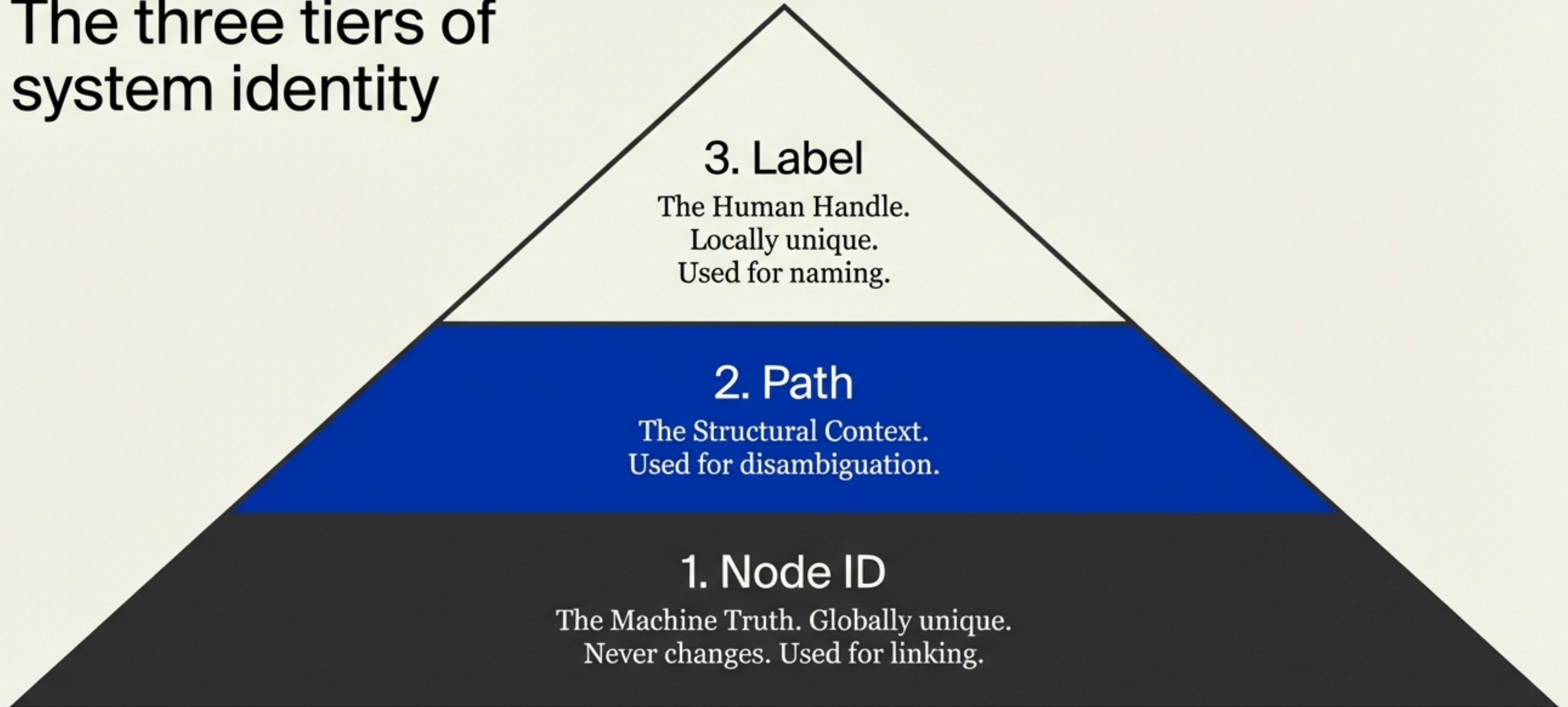
The Rule: Labels must be unique within a folder, but can be duplicated across parallel folders. **Why?** Labels are for human convenience; enforcing global uniqueness on human-readable labels creates bottlenecks.

Disambiguating collisions via Contextual Paths

When labels collide, we use the folder path to distinguish them. Each parent folder maintains an ‘_index.json’ to ensure uniqueness among direct children.



The three tiers of system identity



Strategic decisions on enforcement and persistence



Schema Enforcement

Warn, don't reject (for now)

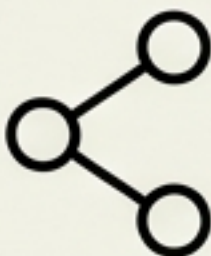
Strict enforcement can hinder development; flexibility is prioritized initially.



Label Persistence

Labels do NOT change on move

Node_ID is the true identity. Changing labels breaks the user's mental model.

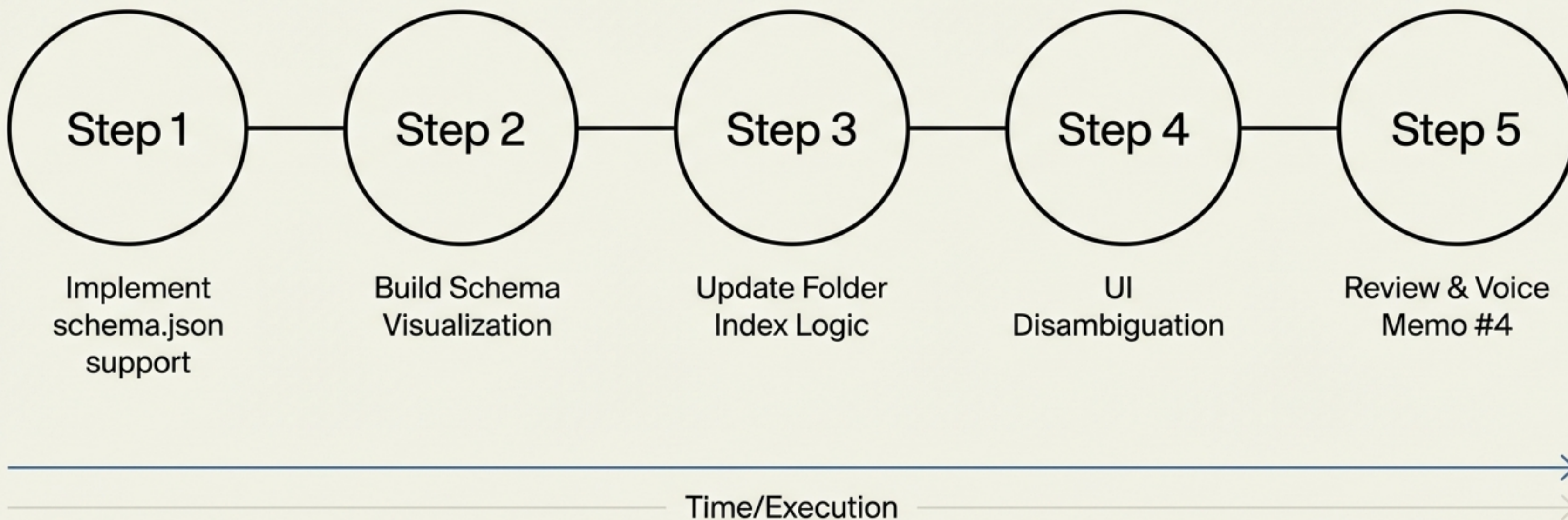


Index Storage

Distributed '_index.json' per folder

Avoids a central index bottleneck and supports the 'self-contained' philosophy.

Implementation Roadmap



A system balancing strict structure with human flexibility

This architecture moves us away from the ‘spaghetti graph’. By treating folders as mini-databases and enforcing fractal order, we create a system that is:

- **Governed:** Visible rules via Meta-Schema.
- **Movable:** Self-contained, portable work units.
- **Navigable:** Context-aware identity and clear paths.

